is to enter them manually (or visited using old bookmarks or external links). This is not a security issue or anything, but what if you want to enforce the new pattern, so that people (and search engines) will be inclined to bookmark and share your URLs this way?

Instinctively, one might modify the *htaccess* file like this:

```
# WRONG! Read below.
RewriteRule ^(.*)\.([a-z][a-z])\.html$
/$2/$1
RewriteRule ^([a-z][a-z])/(.+)$ /$2.$1.
html
```

(In the pattern, a dot means any character while a dot prefixed with a backslash means a literal dot.)

The *htaccess* file shown above is incorrect. First, it results in a loop: the user visits the *.en.html* version of a URL, Apache rewrites it as */en/*, which then gets rewritten as *.en.html*, which goes on and on. It's a great thing Apache detects this and returns 500. (If you are already familiar with the RewriteRule, the *L* flag doesn't help, but *END* does.)

```
$ curl -I localhost/section/subsection/
page.en.html
HTTP/1.1 500 Internal Server Error
```

The second problem is, what we've written here is an internal rewrite, not a redirect. Although we want the rewrite from */en/* to *.en.html* to be invisible, we want the reverse to be a visible and permanent redirect, so that we can enforce our preferred pattern. Thankfully, the solution to this problem automatically solves the first problem (rewrite loop). Here's the modified *htaccess*:

```
# Works fine
RewriteRule ^(.*)\.([a-z][a-z])\.html$
/$2/$1 [R=308]
RewriteRule ^([a-z][a-z])/(.+)$ /$2.$1.
html
```

And here's the *curl* status:

Note:

- CAUTION: Rewrites can facilitate a Directory Traversal Attack when used incorrectly (the attack is possible even without rewrites, but rewrites sometimes increase the possibility). Apache seems to prevent this, but always test to make sure.
- *htaccess* is not the only place where you can put RewriteRule directives. You can use them in your site's main configuration file. Note that you'll have to adapt your pattern based on where the rule is written. For example, global rules can have patterns that start with a slash, while a pattern in *htaccess* is relative to the directory and doesn't have a leading slash.
- The official doc has a lot of details like the above, including flags other than R. Make sure to read it.
- The syntax for URL patterns in RewriteRule is called Regex or Regular Expressions. It has a far wider scope.
- You may have noticed that my site doesn't use the 'clean' scheme described in this article; it's nothing but a personal choice. Also, I don't offer translations for every page, so it doesn't make sense to put languages as URL prefixes.

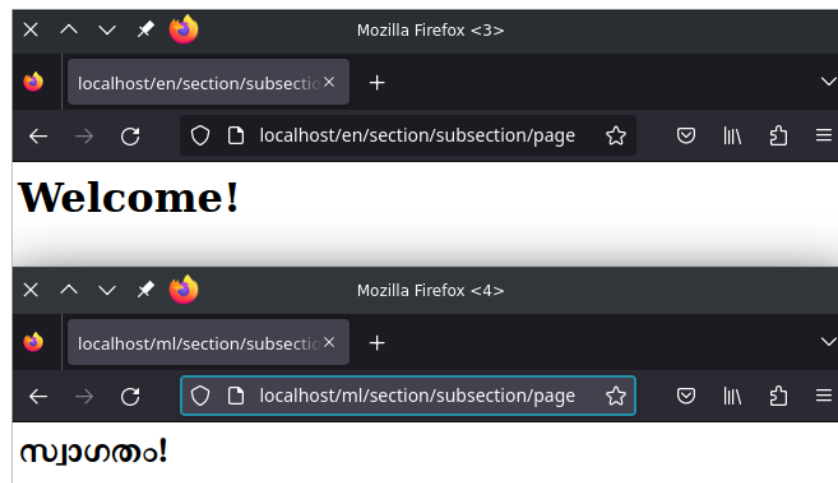

Figure 1: URLs of English and Malayalam pages

```
$ curl -I localhost/section/subsection/
page.en.html
HTTP/1.1 308 Permanent Redirect
...
Location: http://localhost/en/section/
subsection/page
...
```

So, finally, if you try it from a browser, this is what happens now:

1. A */en/* style URL serves the *.en.html* file, without causing any redirect (even the browser doesn't know what's happening behind the scenes).
2. A *.en.html* style URL gets visibly redirected to the */en/*-style URL, and then the above happens. END

By: Nandakumar Edamana

The author is a developer and technology writer who primarily focuses on software freedom, privacy and other ethical issues. He has authored a couple of books and several articles for mainstream publishers. A GNU/Linux user for 15+ years, he releases his software products under free software licences.